

NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 15/05/2026

Task 21 : Use Python libraries for data processing (Pandas, NumPy, Openpyxl).

Objective

The main objective of this task is to use Python libraries like **Pandas, NumPy, and Openpyxl** to efficiently process and manage data. It focuses on reading data from files, cleaning and organizing it, performing numerical operations, and saving the processed data into Excel format.

```
import pandas as pd
import numpy as np
# Using Pandas
df=pd.read_csv('User_Data_Association.csv')
print(df.describe().round())
df.isnull().sum()
df.duplicated().sum()

# Using Numpy
print("Maximum Salary : ",np.max(df['EstimatedSalary']))
print("Minimum Salary : ",np.min(df['EstimatedSalary']))
print("Average Age : ",np.mean(df['Age']))

# Openpyxl

df.to_excel("User_Data_Association.xlsx",index=False)
df2=pd.read_excel("User_Data_Association.xlsx")
df2.head()
```

	User ID	Age	EstimatedSalary	Purchased	
count	400.0	400.0	400.0	400.0	
mean	15691540.0	38.0	69742.0	0.0	
std	71658.0	10.0	34097.0	0.0	
min	15566689.0	18.0	15000.0	0.0	
25%	15626764.0	30.0	43000.0	0.0	
50%	15694342.0	37.0	70000.0	0.0	
75%	15750363.0	46.0	88000.0	1.0	
max	15815236.0	60.0	150000.0	1.0	
Maximum Salary :		150000			
Minimum Salary :		15000			
Average Age :		37.655			
	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

I learned how to use **Pandas** to load, organize, and manipulate data in a structured format like DataFrames. I understood how to clean data, perform basic operations, and analyze it efficiently.

I also learned how to use **NumPy** for fast numerical computations such as calculating mean, maximum, minimum, and applying conditions on data. It helped me understand how mathematical operations can be performed quickly on large datasets.

Additionally, I learned how to use **Openpyxl** to read and write Excel files, which is useful for storing and sharing processed data.

Overall, I gained practical knowledge of how these libraries work together to make data processing faster, easier, and more efficient for real-world applications.

Task 22 : Load and read data from CSV files.

Objective

The main objective of this task is to learn how to import and read data stored in CSV (Comma-Separated Values) files using Python libraries such as Pandas. It

helps in understanding how raw data can be loaded into a structured format (DataFrame) so that it can be easily analyzed, cleaned, and processed.

```
: # Load and read data from CSV files.

df=pd.read_csv("Iris(1).csv")
df.head()
```

	Id	SepalLengthCm	SepalWidthCm
0	1	5.1	3.5
1	2	4.9	3.0
2	3	4.7	3.2
3	4	4.6	3.1
4	5	5.0	3.6

From this code, I learned how to load a dataset from a CSV file into Python using the Pandas library with `pd.read_csv()`. This function helps convert raw CSV data into a structured DataFrame, which makes it easier to analyze and manipulate.

I also learned how to use `df.head()` to quickly view the first few rows of the dataset. This is useful for getting a quick overview of the data, understanding column names, and checking whether the data has been loaded correctly.

Overall, this helps in the initial step of data analysis, where we explore and understand the dataset before performing any cleaning or processing tasks.

Task 23 : Export Data from Google Analytics or Other Tools

Objective

To load and analyze data exported from Google Analytics or other reporting tools using Python and Pandas.

```
# Export Data from Google Analytics or Other Tools
import pandas as pd

df = pd.read_csv(
    "Test_external_data.csv",
    encoding='latin1',
    sep=';',
    on_bad_lines='skip'
)

print(df.head())
print(df.info())
```

```

      Name, Age, City, Marks
0      Ayaan, 22, Pune, 85
1      Rahul, 25, Mumbai, 90
2      Sneha, 23, Delhi, 88
3      Priya, 24, Bangalore, 92
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 1 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name, Age, City, Marks  4 non-null     object
dtypes: object(1)
memory usage: 164.0+ bytes
None
```

First, I used an exported analytics dataset in CSV format. Then, I loaded the dataset into Python using the `read_csv()` function from Pandas.

After loading the dataset, I displayed the first few rows using `head()` to understand the dataset structure. Next, I used `info()` to check: • column names, • data types, • non-null values. This helped me understand how exported analytics data can be imported and analyzed using Python.

During the task, I also handled CSV parsing issues because some exported datasets contain formatting problems or special characters.

Task 24 : Perform time series analysis on the data.

Objective

Analyze data collected over time in order to identify trends, patterns, and changes, and to support future forecasting.

```
df=pd.read_csv('sales_analysis_output.csv')
df.head()
df['Date']=pd.to_datetime(df['Date'])
df
Unit_sold=df.groupby(df['Date'].dt.month)['Units_Sold'].sum()
print(Unit_sold)
```

```
Date
1      933
2      981
3     1080
4     1210
5     1312
6     1466
7     1410
8     1370
9     1260
10     1190
11     1500
12     1848
Name: Units_Sold, dtype: int64
```

I learned how to work with time-based data using Python Pandas. I understood how to load a CSV file and convert the Date column into datetime format so that it can be used for time series analysis.

I also learned how to extract specific time components like month from a date using .dt.month. After that, I learned how to group the data based on months using groupby() and calculate total units sold for each month using sum(). Overall, I understood how to analyze sales trends over time and convert raw data into meaningful monthly insights.

Task 25 : Perform data segmentation.

Objective

The objective of data segmentation is to divide a large dataset into smaller, meaningful groups based on specific features such as age, income, behavior, location, or any other relevant attribute. This helps in simplifying data analysis, identifying patterns, and understanding different categories within the data.

```
# Perform data segmentation.
df=pd.read_csv('User_Data_Association.csv')
df['User_buied']=df['Purchased'].apply(lambda x: "Yes" if x == 1 else "No")
df['Age_check']=df['Age'].apply(lambda x:"Young" if x < 23 else "Adult")
df.head(12)
```

	User ID	Gender	Age	EstimatedSalary	Purchased	User_buied	Age_check
0	15624510	Male	19	19000	0	No	Young
1	15810944	Male	35	20000	0	No	Adult
2	15668575	Female	26	43000	0	No	Adult
3	15603246	Female	27	57000	0	No	Adult
4	15804002	Male	19	76000	0	No	Young
5	15728773	Male	27	58000	0	No	Adult
6	15598044	Female	27	84000	0	No	Adult
7	15694829	Female	32	150000	1	Yes	Adult
8	15600575	Male	25	33000	0	No	Adult
9	15727311	Female	35	65000	0	No	Adult
10	15570769	Female	26	80000	0	No	Adult
11	15606274	Female	26	52000	0	No	Adult

From this code, I learned how to perform data segmentation using Pandas by creating new categorized columns based on existing data. I understood how to load a dataset using `read_csv()` and then transform numerical and binary data into meaningful groups using the `apply()` function with `lambda`.

I learned how to convert the Purchased column into a clear categorical feature (Yes or No), which makes the data easier to interpret for analysis. I also learned how to segment users based on age by classifying them into groups like Young and Adult, which helps in understanding different user types.

Overall, I understood that data segmentation is useful for simplifying raw data, creating meaningful categories, and improving data analysis for better decision-making.

Task 26 : Use Regex (Regular Expressions) for data cleaning.

Objective

The objective of using **Regular Expressions (Regex)** in data cleaning is to remove unwanted characters, extract useful patterns, and standardize messy data. It helps in cleaning text data like names, emails, and phone numbers efficiently, making the dataset more consistent and ready for analysis.

```

# Use Regex (Regular Expressions) for data cleaning
import pandas as pd

data = {
    "Name": ["Ayaan123", "Rahul!!", "Sneha@2024", "Priya#45"],
    "Email": ["ayaan@gmail.com", "rahul@gmail..com", "sneha#mail.com", "priya@gmail.com"],
    "Phone": ["+91-98765-43210", "98765 12345", "Mobile: 88888-77777", "Contact: 99999@66666"]
}

df = pd.DataFrame(data)

df['Clean_Name'] = df['Name'].str.replace(r'^a-zA-Z', '', regex=True)
df['Clean_Email'] = df['Email'].str.replace(r'[a-zA-Z0-9@]', '')
df['Clean_Email'] = df['Clean_Email'].str.replace(r'\.+', '.', regex=True)
df['Clean_Email'] = df['Clean_Email'].str.replace(r'\@+', '@', regex=True)
df['Clean_Phone'] = df['Phone'].str.replace(r'\D', '', regex=True)
df

```

	Name	Email	Phone	Clean_Name	Clean_Email	Clean_Phone
0	Ayaan123	ayaan@@gmail.com	+91-98765-43210	Ayaan	ayaan@gmail.com	91 98765 43210
1	Rahul!!	rahul@gmail.com	98765 12345	Rahul	rahul@gmail.com	98765 12345
2	Sneha@2024	sneha#mail.com	Mobile: 88888-77777	Sneha	sneha#mail.com	88888 77777
3	Priya#45	priya@gmail.com	Contact: 99999@66666	Priya	priya@gmail.com	99999 66666

I learned how to use **Regular Expressions (Regex)** in Python with Pandas to clean and transform messy datasets. I understood how to remove unwanted characters from text fields like names by keeping only alphabets, which helps in standardizing inconsistent data entries.

I also learned how regex can be used to clean email data by removing invalid symbols, correcting repeated dots, and making the format more consistent. This improves data quality and reduces errors in analysis.

In addition, I practiced cleaning phone numbers by extracting only digits from mixed-format strings, which is useful when dealing with real-world unstructured contact data.

Task 27 : Use Pandas and Python for processing JSON data.

Objective

The main objective of this task is to learn how to efficiently work with JSON (JavaScript Object Notation) data using Python and the Pandas library. It focuses on understanding how to load, read, and convert JSON data into a structured tabular format (DataFrame) so that it can be easily analyzed and processed.

```
import pandas as pd

data = [
    {
        "id": 1,
        "name": "Ayaan",
        "age": 23,
        "address": {"city": "Pune", "state": "MH"},
        "marks": [85, 90, 88]
    },
    {
        "id": 2,
        "name": "Rahul",
        "age": 25,
        "address": {"city": "Mumbai", "state": "MH"},
        "marks": [78, 82, 80]
    },
    {
        "id": 3,
        "name": "Sneha",
        "age": 22,
        "address": {"city": "Delhi", "state": "DL"},
        "marks": [92, 89, 94]
    }
]

df=pd.json_normalize(data)
```

```
# Clean & Transform Data

df.rename(columns={"address.city": 'City',"address.state":'State'},inplace=True)

# Average Marks Column

df['Avg_Marks']=df['marks'].apply(lambda x: sum(x)/len(x))

# Average Age
print(df)
print("Average Age:", df["age"].mean())
```

	id	name	age	marks	City	State	Avg_Marks
0	1	Ayaan	23	[85, 90, 88]	Pune	MH	87.666667
1	2	Rahul	25	[78, 82, 80]	Mumbai	MH	80.000000
2	3	Sneha	22	[92, 89, 94]	Delhi	DL	91.666667

Average Age: 23.333333333333332

I learned how to handle JSON (JavaScript Object Notation) data using Python and the Pandas library. I understood how JSON data, which is often semi-structured or nested, can be converted into a structured tabular format using a DataFrame.

I also learned how to load JSON files or JSON data into Pandas using functions like `read_json()` and how to explore and understand the dataset after loading it. In cases of nested JSON, I learned that it may need normalization before analysis.

Task 28 : Convert data into different output formats (e.g., JSON, XML).

Objective

The objective of converting data into different output formats like JSON and XML is to make the data easily shareable, portable, and compatible with different systems. It helps in transforming structured data into standardized formats that can be used in web applications, APIs, and data exchange processes.

```

In [ ]: # Convert data into different output formats (e.g., JSON, XML).

import pandas as pd

data = {
    "Name": ["Ayaan", "Rahul", "Sneha"],
    "Age": [22, 25, 23],
    "City": ["Pune", "Mumbai", "Delhi"]
}

df = pd.DataFrame(data)
# print(df)

# JSON
data_json=df.to_json(orient="records",indent=4)
print("Data Converted to JSON")

# XML
dat_xml=df.to_xml()
print("Data Converted to XML")

Data Converted to JSON
Data Converted to XML

```

I created a dataset using a Python dictionary and converted it into a Pandas DataFrame. Then I transformed this data into two different formats: JSON and XML. I used the `to_json()` function with proper formatting to convert the data into JSON format, and the `to_xml()` function to convert it into XML format.

I learned how to convert structured data into different output formats using Pandas. I understood that JSON format is useful for web applications and APIs, while XML is useful for structured data exchange. I also learned how simple functions like `to_json()` and `to_xml()` make data transformation easy.

Task 29 :Apply statistics to the dataset.

Objective

The main objective of applying statistics to a dataset is to **understand, summarize, and interpret the data effectively.**

```
# Apply statistics to the dataset.  
import pandas as pd  
df=pd.read_csv("student.csv")  
print("Minimum Marks : ",df['Marks'].min())  
print("Maximum Marks : ",df['Marks'].max())  
print("Average of marks : ",df['Marks'].mean())  
  
Minimum Marks : 76.4  
Maximum Marks : 95.2  
Average of marks : 86.775
```

Loaded the dataset from a CSV file using Pandas and explored the **Marks** column by applying statistical functions like `min()`, `max()`, and `mean()`. I calculated the lowest, highest, and average marks to understand the overall performance and distribution of student scores in the dataset.

Learned how to use Pandas to load and analyze a dataset, and how to apply statistical functions like `min()`, `max()`, and `mean()` to understand data. I also understood how these measures help in identifying performance trends and summarizing student results.

Task 30 : Perform basic statistical analysis (mean, median, mode).

Objective

To summarize and understand the dataset by calculating **mean, median, and mode**, which help identify central tendency and overall data distribution.

```
import pandas as pd
df=pd.read_csv("student.csv")
print("Mean of marks : ",df['Marks'].mean())
print("Median of Marks : ",df['Marks'].median())
mode=df['Marks'].mode()
print(mode)

Mean of marks : 86.775
Median of Marks : 88.35
0    76.4
1    78.3
2    85.5
3    88.0
4    88.7
5    90.0
6    92.1
7    95.2
Name: Marks, dtype: float64
```

Loaded the dataset from a CSV file using Pandas and selected the **Marks** column for analysis. I applied statistical functions like `mean()`, `median()`, and `mode()` to calculate the average, middle value, and most frequent marks. This helped me examine the distribution and overall performance of students in the dataset.

Learned how to use Pandas to perform basic statistical analysis on a dataset. I understood the importance of **mean (average performance)**, **median (central value without outlier effect)**, and **mode (most common score)**. I also learned how these measures help in summarizing data, identifying patterns, and making better data-driven decisions.